FinOps Enterprise Technical & Testing Report

Corporate Financial Operations Portal - Northbridge Commercial Bank

Student: Deren Daniel Karabillioglu
Release: 1 of 5
Prepared: October 13, 2026 (UTC)

Purpose and implemented scope

This academic application centralizes corporate client registration and updates, account opening and filtering, read-only transfer investigation, and operational reporting. Business records are loaded through HTTP from JSON persistence; React does not hardcode operational records or totals.

Architecture

Layer	Actual implementation and responsibility
Route	backend/src/routes/index.js declares versioned API endpoints.
Controller	backend/src/controllers/index.js translates HTTP input and sends resource responses.
Service	backend/src/services/index.js applies eligibility, uniqueness, filters and dashboard aggregation.
Repository	backend/src/repositories/ encapsulates asynchronous reads and serialized updates.
Storage	backend/data/ contains clients.json, accounts.json and transactions.json.

Reliability and release boundaries

Updates read the latest existing array inside a per-file promise queue, write a temporary file in the same directory, and rename it over the original. Corrupt or unavailable files produce controlled errors without resetting data. This supports a single backend process; cross-file transactions and multiprocess locking require a database extension. Release 1 deliberately excludes deletion, balance changes, account closure and transaction execution.

Submission status

Both repositories contain the implementation and evidence. Instructor access to the private repository remains to be completed. The assignment requests a private GitHub URL; this PDF is supplementary documentation and does not replace the required repository submission.

02 API contracts and demonstration data

Method	Path (prefix /api/v1)	Purpose
GET	/health	Status and UTC timestamp
GET	/clients	Search, status/industry filters and pagination
POST	/clients	Register an Active client (201)
GET	/clients/:id	Profile and associated accounts
PATCH	/clients/:id	Edit permitted business fields
GET	/accounts	Client/type/currency/status filters
POST	/accounts	Open account for an Active client (201)
GET	/accounts/:id	Account information and owner
GET	/transactions	Combined search, dates, status and account filters
GET	/transactions/:id	Transfer and both related accounts
GET	/dashboard/summary	Seven metrics, chart counts and five recent records

Data model and business validation

Clients have UUIDs, unique trimmed case-insensitive registration numbers, supported industries/countries and valid email addresses. IDs and registration numbers are immutable. Accounts belong to existing clients; only Active clients can receive new accounts. Account numbers are simulated strings. CAD and USD monetary values use safe integer minor units: 1250075 means CAD 12,500.75. Transactions require distinct existing accounts with the same currency.

Dataset	Pristine demonstration records
Clients	10 across 7 industries; 8 Active, 1 Inactive, 1 Suspended
Accounts	20 across CAD and USD; 18 Active, 1 Frozen, 1 Closed
Transactions	50 transfers; 17 Completed, 17 Pending, 16 Rejected

Pagination defaults to 10 and is capped at 100. Date filters include both UTC calendar days, reject impossible dates and reversed ranges, and combine with other filters. Records sort newest first, then ascending transaction UUID for ties. TXN-2026-000001 is a Completed CAD 12.50 supplier payment.

03 Reproduction and verified test evidence

Installation and startup

Use Node.js 22.9+ and npm. In backend: **npm ci**, copy .env.example to .env, run **npm run verify:data**, then **npm start**. In a second terminal, run **npm ci** and **npm run dev** from frontend. Open http://127.0.0.1:5173. Both lockfiles and environment examples are committed.

Backend PORT defaults to 3000. Use FRONTEND_ORIGIN=http://127.0.0.1:5173 with the default frontend command. VITE_API_BASE_URL configures the backend URL. DATA_DIR optionally points to a separate existing dataset. Restarts do not reset stored records.

Verification	Recorded result	Evidence
Backend integration	16 tests passed; 0 failed	evidence/api-tests.txt
Money conversion	3 tests passed; 0 failed	evidence/money-tests.txt
Postman runner	35 requests; 40 assertions; 0 failures	evidence/postman-tests.txt and postman-results.json
Data integrity	10 clients / 20 accounts / 50 transfers verified	evidence/dataset-verification.txt
Production bundle	Vite build completed successfully	evidence/frontend-build.txt
Browser checks	Desktop and tablet flows passed	evidence/browser-tests.txt

What the automated tests establish

Actual HTTP requests cover client creation and updates, account relationships, backend restart persistence, duplicate registrations including concurrent requests, inactive/suspended owners, monetary bounds, combined filters, inclusive dates, deterministic ordering and read-only restrictions. Storage tests preserve corrupt/missing files, serialize 25 updates across two store instances and preserve the dataset after a failed write.

Browser evidence and limits

The browser checks cover navigation, real registration/editing/account creation, confirmation messages, exact balances, dashboard refresh, reference investigation, both related account links, pagination, invalid dates, empty results, tablet overflow and Not Found. Mock responses are used only for loading, error/retry and zero-state checks. CRUD flows use a real backend with disposable copied data. Screenshots may therefore show 11 clients and 19 active accounts after the test registration; the committed pristine data remains 10 and 18 respectively.

Re-running checks

Run **npm test** in backend and frontend, **npm run verify:data** in backend, and **npm run build** in frontend. Import the collection under postman/ with its configurable baseUrl. Browser reproduction instructions and dependencies are in README.md. Postman and browser checks create records, so run them on disposable copies.

04 Technical reflection

1. Layer separation.

routes/index.js declares URLs and methods, controllers/index.js reads request parameters and sends response status/payloads, services/index.js implements uniqueness, client eligibility, filters and aggregate calculations, and repositories/ owns persistence. app.js configures middleware and imports no listener. Putting business logic in routes would couple rules to HTTP and make reuse, isolated verification and future workflow services harder. The shared validation functions provide consistent field-level errors.

2. Concurrent JSON updates.

Two reads of the same initial file can each append a record; the second write would discard the first append. JsonStore.update() chains a promise per absolute file path, then reads the latest dataset inside that chain. A failed operation does not block later updates. The duplicate check occurs inside the update callback. Tests exercise 20 concurrent client registrations and 25 updates across two store instances, plus concurrent duplicate requests. This does not replace cross-process database locking.

3. Minor units.

Binary floating point cannot exactly represent many decimal fractions; multiplying and rounding arbitrary input can accept unwanted precision. frontend/src/lib/money.js validates decimal strings and uses BigInt arithmetic before converting an in-range result to Number. The service independently requires non-negative safe integers, valid currencies and types. Formatting divides integer minor units using BigInt so even the maximum safe amount retains its exact cents. An implementation challenge was ensuring CAD 12,500.75 is sent as 1250075 and rejecting a third decimal place rather than silently rounding it.

4. Async React behavior.

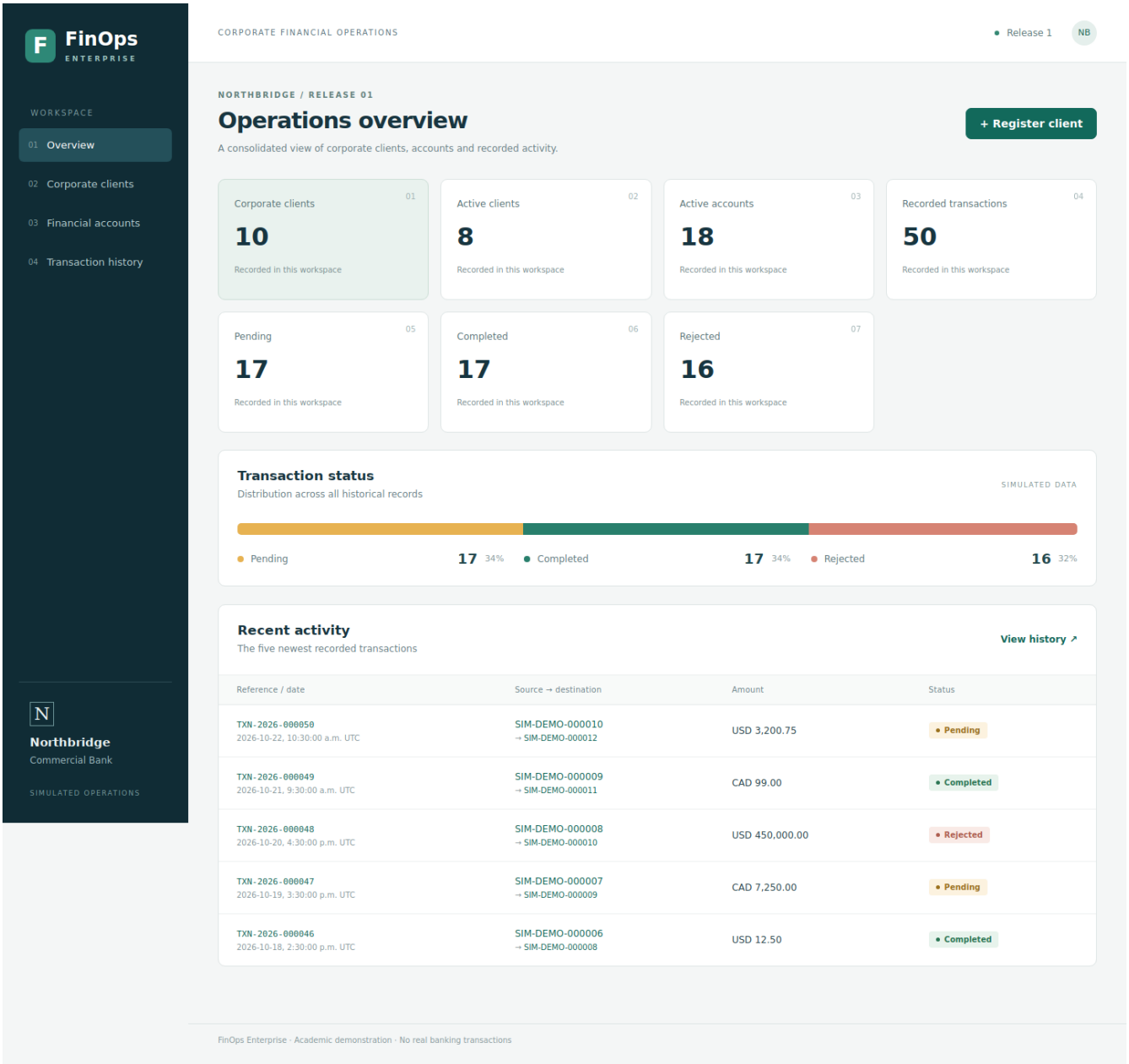
useResource presents loading/error/success states and aborts stale fetches when paths change. The API helper propagates structured field errors and explains network failures. Required inputs, lengths, email type and select options provide preliminary validation. Client and account forms disable their fieldsets during requests and use a synchronous ref guard as well as state to prevent duplicate submissions. They redirect to the resulting details page with a confirmation. Directory filtering resets pagination to page 1. Browser checks verify error, loading, empty and successful states at desktop and tablet widths.

5. Assignment 2 extension.

Reuse the app shell, API client, form fields, error handling, entity repositories and validation helpers. Add transaction initiation forms, transaction processing routes/controllers/services, lifecycle rules, approval entities and state transitions. Transfer execution must atomically update two balances and a transaction record; the existing per-file queue cannot guarantee that. Move persistence behind the repository interfaces to a transactional database, introduce idempotency keys and concurrency control, and test pending/completed/rejected transitions. Keep historical GET handlers side-effect free. Add authentication and roles in the release prescribed by the course rather than assuming this first release is secure for real operations.

05 Dashboard evidence

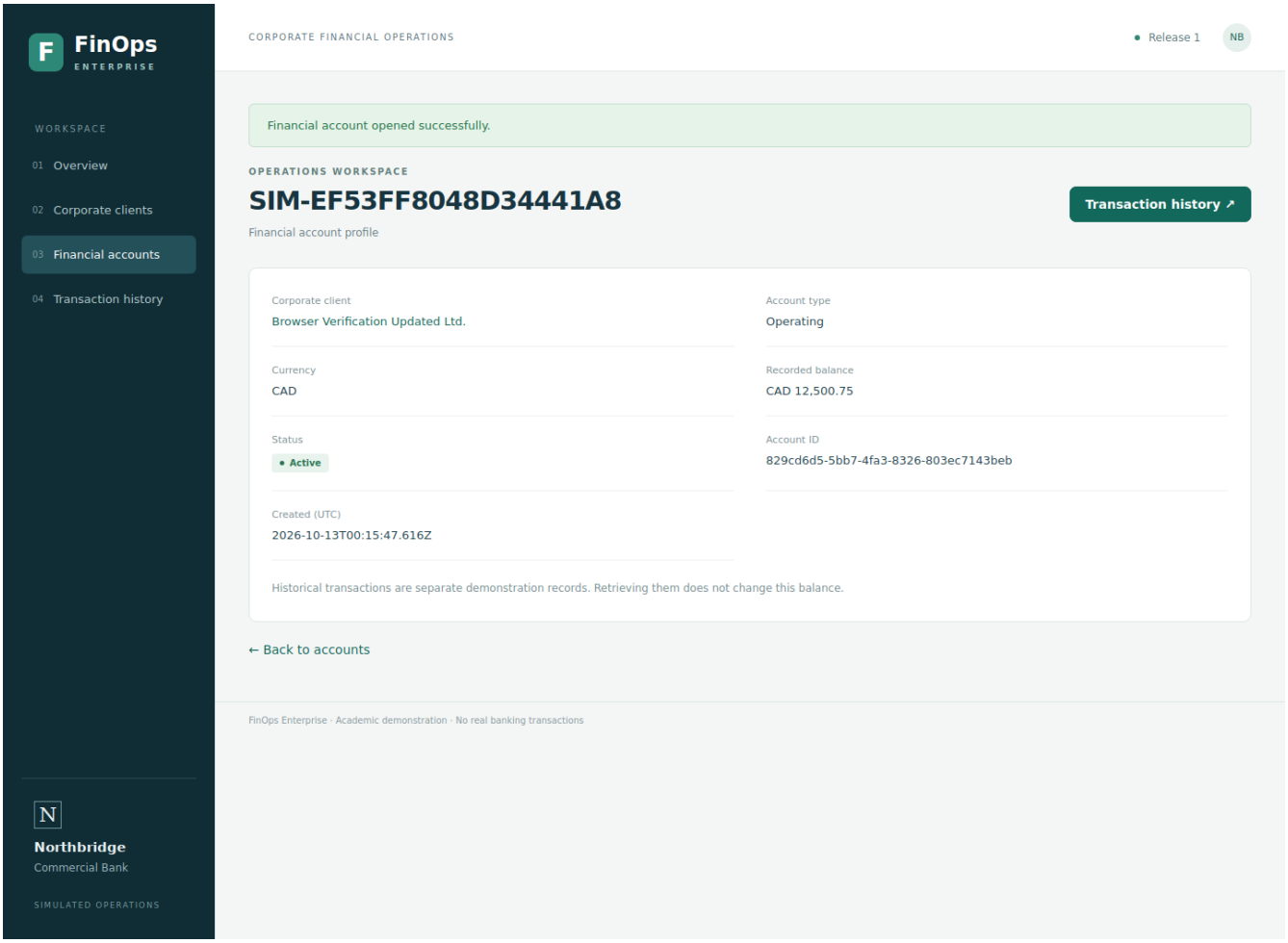
Desktop dashboard from the pristine dataset. The seven indicators and status chart agree with the backend data. The recent table shows five transfers ordered from newest to oldest.



Original full-resolution file: evidence/01-dashboard-desktop.png

06 Account registration evidence

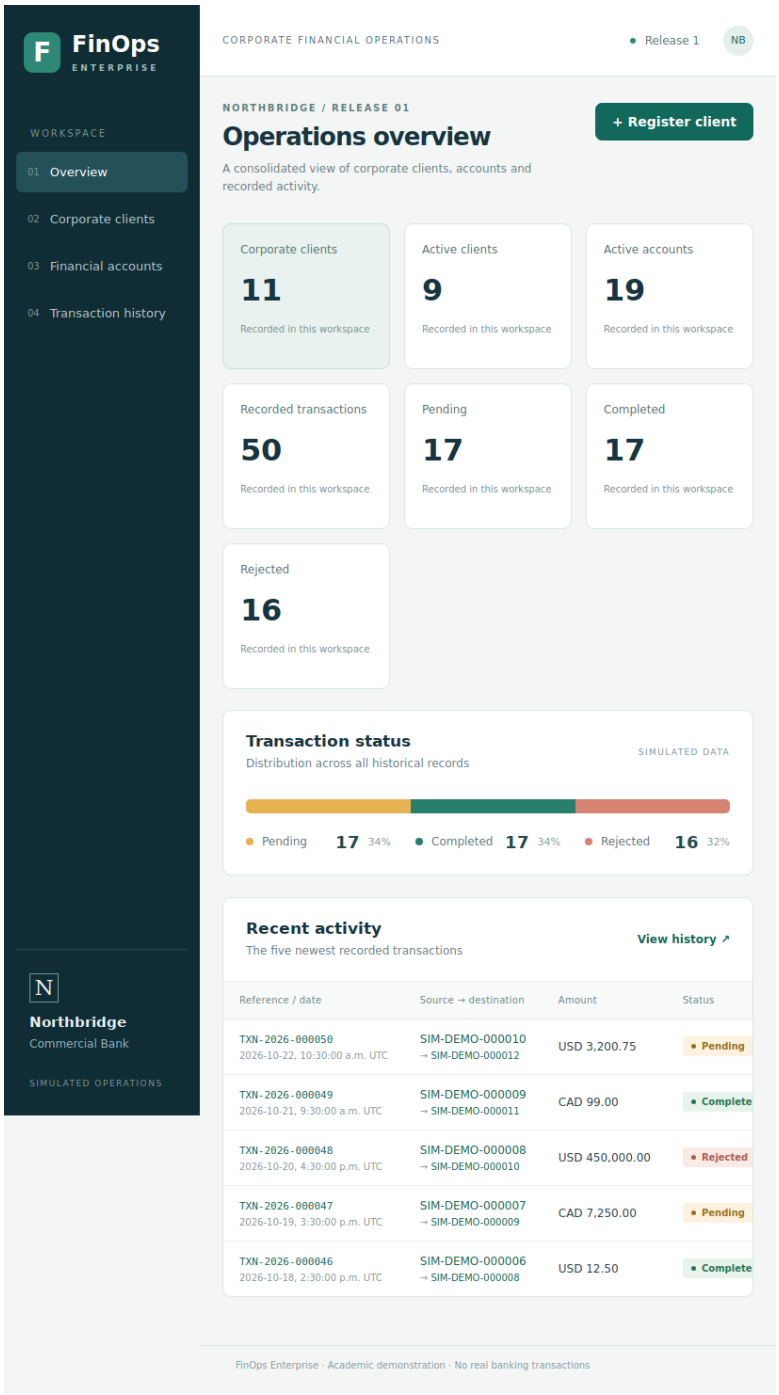
A real browser workflow created an account for a newly registered client and displayed CAD 12,500.75 exactly. The confirmation, account ownership, status and historical transaction navigation were verified.



Original full-resolution file: evidence/04-account-details.png

07 Tablet and accessibility evidence

Tablet viewport: 820 x 1180. This image follows the browser test client and account registration, so totals are 11 clients and 19 active accounts. The test confirmed working navigation and no horizontal overflow of the entire page.



Original full-resolution file: evidence/07-dashboard-tablet.png

All 10 original screenshots are included under evidence/, including client directory/form, transaction details/history, account directory, controlled API error and empty-dashboard views.